

Les 8 projets suivants seront réalisés sous forme de notebooks Jupyter. Chaque fiche décrit le problème, le contexte, les éléments théoriques utiles, et la liste des tâches à accomplir. Vous devez produire un notebook structuré contenant explications, code, résultats et interprétations.

Chaque notebook doit contenir :

1. Une introduction présentant le problème et le contexte.
2. Une section de rappels mathématiques et numériques.
3. Des cellules de code commenté, avec explications des choix.
4. Des graphiques et tableaux illustrant les résultats.
5. Une conclusion interprétant les résultats et discutant des limites.
6. Des questions de réflexion pour approfondir.

Bon courage et bonne modélisation !

Projet 1 : Résolution numérique d'équations non linéaires

Problème et situation

On cherche à déterminer les racines d'équations non linéaires de la forme $f(x) = 0$ issues de modèles physiques ou économiques.

Situation type : Calcul de la profondeur d'enfoncement d'un pieu dans le sol modélisée par $f(x) = x^3 - 5x^2 + 2x - 1$. Aucune solution analytique n'existe.

Rappels théoriques

- La méthode de Newton-Raphson : $x_{n+1} = x_n - f(x_n)/f'(x_n)$.
- La méthode de la sécante (sans dérivée).
- L'algorithme hybride de Powell implémenté dans `scipy.optimize.fsolve`.
- Conditions de convergence : continuité, changement de signe, dérivée non nulle.

Tâches à réaliser

1. Définir la fonction $f(x)$ donnée et tracer son graphe sur un intervalle pertinent pour localiser les racines visuellement.
2. Utiliser `scipy.optimize.fsolve` pour trouver une racine proche d'un point initial donné.
3. Implémenter manuellement la méthode de Newton et comparer sa convergence avec `fsolve`.
4. Tester sur une deuxième équation de votre choix (par exemple $g(x) = \cos(x) - x$).
5. Afficher un tableau récapitulatif des différentes approximations et du nombre d'itérations.
6. Discuter la sensibilité au choix de la condition initiale.

Projet 2 : Simulation de systèmes dynamiques (EDO)

Problème et situation

Modéliser l'évolution temporelle d'un système dynamique régi par des équations différentielles ordinaires.

Situation type : Modèle proie-prédateur de Lotka-Volterra :

$$\frac{dx}{dt} = \alpha x - \beta xy, \quad \frac{dy}{dt} = \delta xy - \gamma y.$$

Rappels théoriques

- Équilibres : $(0, 0)$ et $\left(\frac{\gamma}{\delta}, \frac{\alpha}{\beta}\right)$.
- Cycles périodiques autour du point d'équilibre non trivial.
- Solveur `solve_ivp` : méthodes RK45, Radau, BDF.

Tâches à réaliser

1. Définir la fonction dérivée du système de Lotka-Volterra avec paramètres $\alpha, \beta, \delta, \gamma$.
2. Résoudre le système sur $t \in [0, 30]$ avec une condition initiale $(x_0, y_0) = (10, 5)$.
3. Tracer les évolutions temporelles des deux populations.
4. Tracer le portrait de phase $(x(t), y(t))$.
5. Faire varier un paramètre (par exemple α) et observer les changements de comportement.
6. Comparer les résultats avec différentes méthodes numériques (RK45 vs Radau).

Projet 3 : Programmation linéaire (Simplexe)

Problème et situation

Optimiser une fonction linéaire sous contraintes linéaires.

Situation type : Une entreprise fabrique deux produits x et y avec des ressources limitées (heures machine, matières premières). Maximiser le profit $P = 3x + 5y$ sous :

$$x + 2y \leq 40, \quad 3x + y \leq 60, \quad x \geq 0, y \geq 0.$$

Rappels théoriques

- Forme standard d'un programme linéaire.
- Méthode du Simplexe : parcours des sommets du polytope.
- Dualité et analyse de sensibilité.
- Fonction `linprog` de `scipy.optimize`.

Tâches à réaliser

1. Formuler mathématiquement le problème de maximisation.
2. Résoudre graphiquement (tracer les contraintes, région réalisable, courbes de niveau).
3. Utiliser `scipy.optimize.linprog` pour obtenir la solution optimale.
4. Implémenter une version simple de l'algorithme du Simplexe (tableaux).
5. Comparer les résultats avec la méthode graphique.
6. Ajouter une troisième variable et une contrainte, résoudre à nouveau numériquement.

Projet 4 : Création de notebooks interactifs pour concepts mathématiques

Problème et situation

Développer un notebook pédagogique interactif pour expliquer un concept mathématique.

Situation type : Visualisation des séries de Fourier ou de la convergence de suites.

Rappels théoriques

- Les widgets interactifs `ipywidgets` (sliders, boutons, menus).
- Représentation graphique dynamique avec `matplotlib`.
- Concept au choix : convergence de série de Fourier, méthode de Newton, intégration de Riemann, etc.

Tâches à réaliser

1. Choisir un concept mathématique de niveau master.
2. Écrire une introduction théorique concise.
3. Créer des widgets interactifs permettant de modifier des paramètres (ex. nombre d'harmoniques pour Fourier, pas d'intégration, paramètre de fonction).
4. Afficher une animation ou un graphique qui se met à jour en temps réel.
5. Ajouter des explications textuelles qui changent selon les paramètres.
6. Tester l'utilisabilité du notebook par un autre étudiant.

Projet 5 : Optimisation non linéaire avec contraintes

Problème et situation

Résoudre un problème d'optimisation non linéaire avec contraintes d'inégalité.

Situation type : Concevoir un réservoir cylindrique de volume $V = 1 \text{ m}^3$ à moindre coût de matière (surface minimale). Contrainte : hauteur h et rayon r positifs.

Rappels théoriques

- Fonction coût : $S(r, h) = 2\pi r^2 + 2\pi r h$.
- Contrainte : $\pi r^2 h = 1$.
- Multiplicateurs de Lagrange, conditions KKT.
- `scipy.optimize.minimize` avec méthodes SLSQP, trust-constr.

Tâches à réaliser

1. Définir la fonction objectif (surface) et la contrainte d'égalité.
2. Résoudre le problème analytiquement par substitution et comparer.
3. Utiliser `minimize` avec une condition initiale arbitraire.
4. Tester différentes méthodes d'optimisation (SLSQP, COBYLA).
5. Ajouter une contrainte supplémentaire : $h \leq 2r$ et observer la nouvelle solution.
6. Visualiser la convergence (valeur de l'objectif en fonction des itérations).

Projet 6 : Analyse et interpolation de données scientifiques

Problème et situation

Approximer des données bruitées ou reconstruire une fonction inconnue à partir de points épars.

Situation type : Mesures expérimentales d'une température en fonction du temps (données bruitées).

On souhaite lisser et interpoler.

Rappels théoriques

- Interpolation polynomiale (Lagrange, Newton) et ses limites (phénomène de Runge).
- Splines cubiques (`scipy.interpolate.CubicSpline`).
- Lissage par moindres carrés (`numpy.polyfit`).

Tâches à réaliser

1. Générer des données simulées à partir d'une fonction connue $f(t) = \sin(t) + 0.1 \times \text{bruit}$.
2. Appliquer une interpolation polynomiale de degrés variés (1, 5, 15) et commenter l'erreur.
3. Utiliser une spline cubique et comparer avec l'interpolation polynomiale.
4. Tester un lissage polynomial (degré 3) par moindres carrés.

Projet 7 : Méthode des éléments finis 1D simple

Problème et situation

Résoudre une équation différentielle du second ordre sur un domaine unidimensionnel par la méthode des éléments finis.

Situation type : Équation de Poisson 1D avec conditions aux limites de Dirichlet :

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = 0, \quad u(1) = 0.$$

Par exemple $f(x) = \sin(\pi x)$.

Rappels théoriques

- Formulation variationnelle : trouver $u \in H_0^1$ telle que $\int_0^1 u'v' = \int_0^1 fv$ pour toute fonction test v .
- Discrétisation par éléments finis linéaires (fonctions chapeau).
- Assemblage de la matrice de rigidité et du vecteur second membre.
- Utilisation de `scipy.sparse.linalg.spsolve` pour la résolution.

Tâches à réaliser

1. Discrétiser l'intervalle $[0, 1]$ en N sous-intervalles égaux.
2. Construire la matrice de rigidité (tridiagonale) et le vecteur second membre.
3. Résoudre le système linéaire obtenu.
4. Comparer la solution numérique avec la solution exacte (quand disponible).
5. Étudier l'ordre de convergence : tracer l'erreur en fonction du pas h (échelle log-log).